

Counting Fruits with State-of-the-Art Multi-View-Stereo Approaches

Bachelor Thesis in Computer Science

presented by

Alexander Link

born on 29.08.2004 in Bamberg

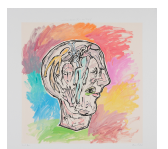
submitted to

Department of Computer Science
Professorship for Cognitive Computer Vision
Friedrich-Alexander-Universität Erlangen-Nürnberg

Academic Supervision: Prof. Dr. Bernhard Egger

Begin date: 01.05.2026

End date: 01.10.2026



Abstract

Contents

1	Introduction	1
1.1	Challenges	1
1.2	Our Approach	2
1.3	Contributions	2
2	Related Work	3
2.1	Fruit Counting and Yield Estimation	3
2.2	Multi-View Instance Association and 3D Segmentation	4
3	Fundamentals	5
3.1	VGGT	5
3.1.1	Context	5
3.1.2	Input Data	6
3.1.3	Pipeline	6
3.1.4	Output	7
3.1.5	Advantages and limitations	7
3.2	Graph Theory	8
3.2.1	Basic concepts	8
3.2.2	Union-Find	9
3.3	Hungarian Matching	9
3.3.1	The Assignment Problem	9
3.3.2	The Hungarian Algorithm	10
3.3.3	Partial Matching	11
3.4	RANSAC	11
3.5	Clustering	12
3.5.1	Density-Based Clustering	12
3.5.2	Agglomerative Clustering with complete Linkage	12
4	Method	13
5	Results	15
6	Conclusions	17

CONTENTS

Chapter 1

Introduction

Feeding a growing world population is a major global challenge, and projections estimate that total food demand will rise substantially over the coming decades [7]. Precision agriculture responds to this by using advanced technology and data analysis to maximize crop yields while reducing waste and environmental impact [17]. Within precision agriculture, counting the fruit on a tree is an important building block: it provides the yield estimates that growers rely on to plan the harvest and allocate labor, storage, and other resources [27]. Because performing the count manually across many trees is slow and labor-intensive, automating it from images is a valuable capability.

Among the ways to automate fruit counting, reconstructing the tree in 3D from a set of RGB images is particularly attractive, for two reasons. First, lifting the count into a shared three-dimensional representation lets each fruit be counted exactly once, instead of being counted repeatedly across the many overlapping images in which it appears. Second, capturing the necessary data requires only an ordinary color camera, of the kind built into any consumer device, rather than the specialized and more expensive depth-sensing equipment, such as LiDAR scanners or calibrated stereo rigs, on which other 3D-capture methods depend. This makes the approach inexpensive and easy to deploy in the field.

citation for cost, other 3D capture methods

Recent work such as FruitNeRF [25] shows that image-based 3D fruit counting is feasible by training a Neural Radiance Field [26] per scene and counting the fruit in the resulting volumetric representation. This approach is effective, but it requires a time-consuming optimization for every new scene. In this work, we instead investigate whether a feed-forward multi-view stereo model, VGGT [33], can replace this per-scene training. Rather than optimizing a representation for each scene, VGGT predicts a 3D reconstruction in a single forward pass. This promises a significant reduction in runtime, but raises the central question of whether the resulting geometry is accurate enough for reliable counting.

1.1 Challenges

Counting fruit reliably from images of a tree faces a number of difficulties, some inherent to the trees themselves and some arising from the way they must be captured.

The trees are difficult subjects in their own right. Fruit are frequently occluded, both by foliage and by other fruit, so that any single image captures only part of the crop. Their appearance changes

with the viewing angle and with lighting conditions such as direct sunlight or shadow, which makes the same fruit look different when seen from different viewpoints. In addition, the branches, leaves and fruit of a tree canopy form a geometrically complex, cluttered structure that is hard to reconstruct consistently.

The way the tree is captured adds further difficulties. To limit occlusion and avoid missing fruit, a tree should be observed from all sides. The number of images that can be used per tree is bounded, however, by the memory available to the reconstruction model. (see Section 3.1.5). To still achieve full coverage, the cameras must therefore be placed sparsely around the tree, spanning a full 360-degree range. As a consequence, neighboring images are taken from widely spaced viewpoints and often show markedly different regions of the tree, with little visual content in common. This makes it hard to establish *correspondences* between images, that is, to determine which parts of one image show the same physical points as another. The problem is compounded by the scene not being perfectly static during capture: wind moves branches, leaves, and fruit between one image and the next, so that even points that are visible in both images no longer align, further degrading the cross-view correspondences on which the later counting steps rely. These unreliable correspondences are the root cause of the counting errors this thesis addresses: depending on the situation, the same apple may be split into several counts or several apples merged into one. How these errors arise in our pipeline, and how we counteract them, is discussed in Chapter 4.

1.2 Our Approach

To count the fruit in a scene, our method maps semantic information, in the form of 2D instance masks obtained with SAM3 [3], across views by means of the 3D representation predicted by VGGT. The association of masks across views follows the pointmap-based principle introduced by CCGS [16]: pixel correspondence between views are established through the predicted pointmaps, and instance masks are matched across views by solving an assignment problem that allows partial matches. We adopt this association step, transfer it from the image-pair setting of its origin to the multi-view reconstruction produced by VGGT, and build a new counting pipeline on top of it. In this pipeline, the pairwise mask matches are aggregated into multi-view instances through a graph formulation, and the fruit are counted by combining these associated instances with a subsequent clustering step.

1.3 Contributions

This thesis makes the following contributions. We investigate whether the feed-forward multi-view stereo model VGGT can replace per-scene optimization for image-based fruit counting, and we assess whether its predicted geometry is accurate enough for this task. Building on the pointmap-based mask association introduced by CCGS [16], we develop a fruit-counting pipeline that extends this association with a graph-based aggregation of instances across many 360-degree-views, sphere fitting as an apple model, and a final clustering step to obtain the count. We further propose an iterative refinement of this pipeline that detects and resolves over-merged instances, thereby addressing the opposing split-and-merge errors that arise under sparse, widely spaced views. Finally, we evaluate both the non-iterative and the iterative variant on

real-world
data?

Chapter 2

Related Work

2.1 Fruit Counting and Yield Estimation

Methods for counting fruit differ mainly in where the counting takes place: in the individual image, across a sequence of images, or in a three-dimensional representation of the scene. These stages can be read as increasingly robust answers to the core difficulty of counting across views, namely avoiding that the same fruit is counted more than once.

The earliest and simplest methods operate purely in the image plane, separating the task into detecting fruit in each image and counting the detections [12]. Detection has moved from handcrafted color and shape features to deep neural networks [27], and modern detectors such as YOLO reach high accuracy on individual images [1]. Because these methods count on a per-frame basis, however, they have no notion of which detections in different images correspond to the same fruit, so a fruit visible in several frames is counted several times.

To resolve this, a second line of work tracks fruit across consecutive frames. Liu et al. [22] proposed the first complete pipeline of this kind, using Structure-from-Motion (SfM) to link detections across frames and correct the resulting double counts, and recent pipelines pair modern detectors with dedicated tracking modules for the same purpose [4]. In later work, Liu et al. [23] matched detections between frames with the Hungarian algorithm on top of a semantic SfM reconstruction, using a single camera. What these methods share is a reliance on densely sampled video: consecutive frames differ only slightly, so detections can be tracked through strong frame-to-frame correspondences, and the association is typically driven by 2D image-space cues such as bounding-box overlap or optical flow between adjacent views. Our setting is the opposite. To cover a tree from all sides within a limited view budget, the cameras are placed sparsely around a full 360-degree range, so neighboring views often show markedly different regions of the tree and share little image-space overlap. Frame-to-frame tracking in the image plane is therefore not applicable, and the association must instead be established in 3D, through the reconstructed geometry rather than through 2D overlap. We adopt the Hungarian algorithm for this association as well, but apply it to correspondences derived from the 3D reconstruction rather than 2D bounding-box overlap between adjacent frames.

A third line of work removes double counting structurally by moving the count into three dimensions, so that each fruit exists once in a shared spatial representation. Häni et al. [12] reconstructed rows of trees from both sides and merged the counts from the combined 3D models. FruitNeRF [25] takes

this idea furthest by integrating the counting directly into a neural scene representation, unifying reconstruction and segmentation that earlier methods had kept separate. Its drawback is that the neural representation must be optimized for each scene, which is computationally expensive; this cost is the immediate starting point of the present work. Newer methods reduce it by replacing the neural field with faster representations, such as the 3D Gaussian splatting used by FruitGaussian [15] and CountingFruit [21], but they remain tied to a per-scene optimization. A different way to avoid this optimization is to associate instance masks across views through a 3D representation rather than to optimize one; since this is the strategy of the present work, we treat it separately in the next section.

2.2 Multi-View Instance Association and 3D Segmentation

Associating 2D instance masks across many views into consistent 3D instances is a problem closely related to fruit counting, and one that general-purpose 3D scene-understanding methods address independently of any counting application. What distinguishes these methods is the criterion by which they decide that two masks seen in different views from the same object.

The most direct criterion is geometric overlap in an explicit reconstruction. Gené-Molá et al. [11] segment fruit with Mask R-CNN, project the 2D masks into an SfM point cloud, associate detections by their overlap in 3D, and discard false detections with a support vector machine. The association is accurate but inherits the cost of the underlying SfM reconstruction, which the authors note is computationally intensive and unsuitable for real-time use.

A second criterion replaces this geometric overlap with cross-view agreement. Maskclustering [36] builds a global graph whose nodes are the 2D masks and weights each edge by a *view consensus rate*, the fraction of masks from other views that contain both endpoints; instances then emerge from iteratively clustering the masks that agree most strongly. The decision is thereby made globally, over all views at once, rather than only between neighboring frames, but it rests on how masks co-occur across views rather than on explicit 3D correspondences.

The criterion our method builds on, introduced by CCGS [16], derives the association directly from dense pointmaps. Pixel correspondences between views are found by minimizing the Euclidean distance between the predicted pointmaps; the overlap between two masks is redefined in terms of these correspondences; and the resulting matching cost is solved with the Hungarian algorithm, with partial matches permitted. CCGS uses the associated masks to build a compact segmentation field for 3D Gaussian splatting, and it operates on image pairs from a DUST3R reconstruction. We take this pointmap-based association and, instead, apply it to the joint multi-view reconstruction produced by VGGT, using it as the first stage of our counting pipeline.

Chapter 3

Fundamentals

3.1 VGGT

The central component of the reconstruction stage in this work is VGGT (Visual Geometry Grounded Transformer), a feed-forward transformer introduced by Wang et al. [33] and awarded the Best Paper Award at CVPR 2025. In this section, we first contextualize VGGT within the broader field of 3D reconstruction and then examine its input, data, internal pipeline, and outputs. Finally, we discuss the properties that make it a suitable foundation for our coupling method.

3.1.1 Context

Reconstructing the three-dimensional structure of a scene from a set of two-dimensional images is a long-standing problem in computer vision [13]. The classical solution is a multi-stage pipeline consisting of Structure-from-Motion (SfM) and Multi-View Stereo (MVS). In the implementation by Schönberger and Frahm [31], SfM estimates the camera poses as well as a sparsely populated point cloud by extracting and matching handcrafted features across the images, followed by an iterative optimization process known as bundle adjustment. A subsequent MVS step then densifies this reconstruction. While these methods are accurate and remain widely used, they rely on iterative optimization that is computationally expensive and can take several minutes to hours per scene to complete.

More recently, learning-based approaches have replaced parts of this pipeline with neural networks. Neural Radiance Fields (NeRFs) [26] represent a scene as a volumetric radiance function encoded in the weights of a neural network, while 3D Gaussian Splatting [18] models it as a collection of colored Gaussian distributions. Both produce high-quality reconstructions but require separate optimization for each individual scene, which again incurs a substantial runtime cost per reconstruction.

A different line of work aims to eliminate per-scene optimization entirely by predicting geometry with a neural network. DUST3R [34] pioneered this *feed-forward* principle by regressing a dense pointmap from an image pair in a single network pass, followed by MAST3R [20], which added feature mapping. However, these methods operate on image pairs and still require a global alignment step to fuse the pairwise predictions into a consistent scene. VGGT [33] belongs to this feed-forward family, but generalized it from pairs to arbitrary sets of images: It processes one, a few, or up to hundreds of views together in a single forward pass and predicts a globally consistent reconstruction without any

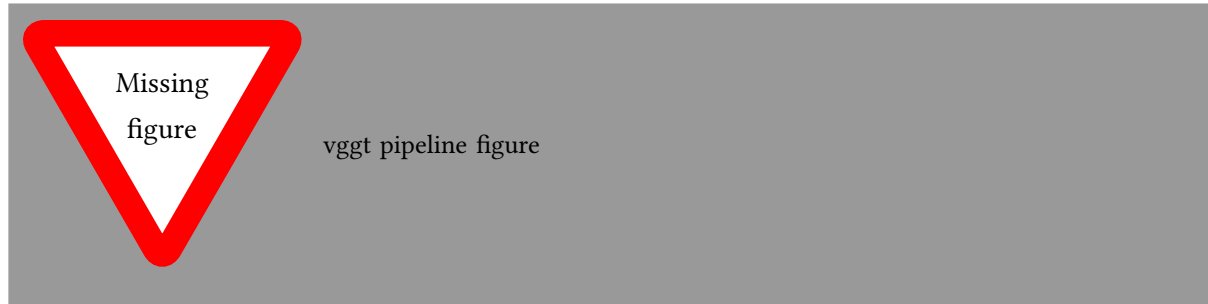
subsequent optimization. This combination of speed and multi-view consistency motivates its use in this work, where fast reconstruction is a central goal.

3.1.2 Input Data

The input for VGGT is a sequence of N RGB images $(I_i)_{i=1}^N$ with $I_i \in \mathbb{R}^{3 \times H \times W}$, which capture the same static scene. Unlike the classical SfM pipeline, no camera parameters, calibration data, or prior pose information need to be provided: The model derives all geometric quantities directly from the image content. The number of views N is flexible and can range from a single image to several hundred, with the only practical limitation being the available GPU memory, since all views are processed jointly [33].

3.1.3 Pipeline

Internally, VGGT maps the input sequence through three conceptual stages to a set of dense 3D predictions: tokenization, alternating attention, and task-specific prediction heads. An overview is shown in Figure.



Tokenization. Each input image I_i is first divided into a grid of non-overlapping image patches. Each patch is then encoded into a *token*, a fixed-length vector that represents the local content of that patch and serves as the basic unit on which the network operates [8]. In VGGT, this patch encoding is performed by a pretrained DINOv2 backbone [29]. An image is thus converted into a set of tokens, each of which carries the local appearance and structural information of its region. For each image, the patch tokens are augmented with additional learnable tokens: a *camera token*, which aggregates the information needed to predict the camera parameters of this image, as well as a set of *register tokens*, which serve as auxiliary memory during the attention calculation and are discarded before the final prediction.

Alternating Attention. The core of the network is a transformer [32] that processes all tokens of all images jointly. It consists of 24 attention blocks that alternate between two types of self-attention. In *frame-wise* self-attention, tokens attend only to other tokens of the same image, which preserves and refines the local structure of each image. In *global* self-attention, all tokens attend to one another across all images, thereby exchanging information between views and establishing the correspondences that link the individual images into a single, coherent scene. By alternating between these two modes, the network balances local detail with global multi-view consistency. This alternating attention design is the central architectural mechanism of VGGT; the authors show that it is superior to both purely global attention and cross-attention alternatives [33].

Prediction Heads. After the attention blocks, the refined tokens are passed to specialized prediction heads. A dedicated *camera head* processes the camera tokens to regress the intrinsic and extrinsic camera parameters of each view. A *DPT head* [30], a Dense Prediction Transformer, upsamples the image tokens back to the input resolutions to generate the dense, pixel wise outputs, namely depth maps, point maps, tracking features, and the corresponding confidence maps.

3.1.4 Output

For each input iamge I_i , VGGT predicts the following quantities:

- camera extrinsics and intrinsics of the view,
- a dense depth map together with a per-pixel confidence map,
- a dense point map together with a per-pixel confidence map, and
- dense feature maps used for tracking pixels across views.

The point map is the output most relevant to this work. It assigns a 3D scene coordinate $P_i(y) \in \mathbb{R}^3$ to every pixel y of an image, thereby representing a dense, pixel-wise 3D reconstruction. Crucially, the point maps of all images are expressed in a single, common coordinate system: they are defined in the reference frame of the first camera, which serves as the world origin [33]. Points observed in different views therefore already lie in a common 3D space without requiring an explicit alignment or registration step. This shared coordinate frame is the property our method builds upon, as it allows 2D instance masks from different views to be related to one another purely through their 3D coordinates.

Each dense prediction is accompanied by a confidence map that estimates the pixel-wise reliability of the reconstruction. These confidence values provide a signal for identifying and filtering out unreliable regions, such as poorly observed or geometrically ambiguous areas [33].

3.1.5 Advantages and limitations

The main advantage of VGGT for our application is its speed. Since the entire reconstruction is generated in a single forward pass, a scene can be reconstructed in a matter of seconds, whereas the SfM-based and scene-specific optimization approaches discussed in Section 3.1.1 take minutes to hours. At the same time, according to the authors, VGGT matches or exceeds the accuracy of these optimization-based alternatives across multiple 3D tasks [33], and does so without any post-processing. Another practical advantage is that it operates directly on uncalibrated images and outputs camera parameters, depth, and point maps together, so that no separate calibration or pscoe estimation step is required.

The fundamental limitationis that all input views are processed collectively via global self-attention, whose memory requirements increase with the number of tokens and, consequently, with the number of input views. The authors note that a naive implementation can become very memory intensive when dealing with a large number of tokens, so that in practice, the available GPU memory limits the number of images that can be reconstructed together [33]. This constraint is directly relevant to our scenario, in which a fruit tree must be captured from all sides: The limited view budget forces a sparse distribution of cameras around the scene, which in turn weakens the correspondences between neighboring views. A number of follow-up works address this scaling limit, for example by processing long sequences in

segments, as in VGGT-Long [6] and VGGT-SLAM [24], or by scaling up to thousands of images as in Fast3R [37]; Pi3 [35] additionally eliminates the dependence on a fixed reference view. These extensions are beyond the scope of this work, which uses the original VGGT model.

3.2 Graph Theory

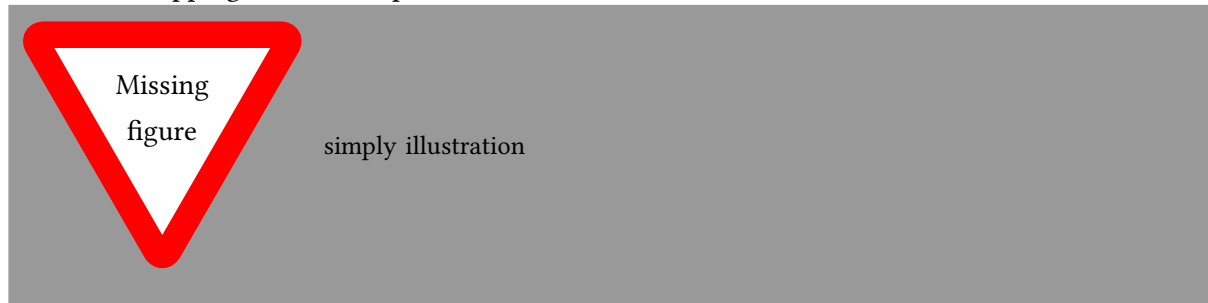
Once VGGT has placed the pixels of all views into a common 3D coordinate frame, the reconstruction stage is followed by the task of deciding which 2D instance masks across the different views belong to the same physical apple. We formulate this assignment as a problem on a graph. In this section, we introduce the graph theory concepts used in our method and define them directly in the context of our problem. Building on this, we describe the "union-find" data structure, which we use to extract the corresponding instances from the graph.

3.2.1 Basic concepts

A graph $G = (V, E)$ consists of a set of vertices (or nodes) V and a set of edges $E \subseteq V \times V$, where each edge connects a pair of vertices. Two vertices are said to be *adjacent* if they are connected by an edge. A *path* is a sequence of vertices in which every consecutive pair of vertices is connected by an edge. A *connected component* is a maximal subset of vertices in which every pair of vertices is connected by a path and which is not connected to any vertex outside the subset. Intuitively, a connected component is a group of nodes that are all directly or indirectly connected to one another, while being separated from the rest of the graph.

We apply these concepts to our association task as follows. Each *node* corresponds to a single 2D instance mask in a single view, i.e. a pair (i, m) consisting of an image index i and an instance identifier m within that image. An *edge* between two nodes expresses the hypothesis that the two masks depict the same physical apple, observed from two different views. How these edges are established, namely by matching instances between image pairs, is described in Section 3.3. A *connected component* then corresponds to a single apple: it groups together all instance masks across all views that were transitively linked to one another and therefore presumably depict the same fruit.

The last property is what makes the graph formulation useful for our purpose. Edges are only ever created between pairs of views, but an apple is typically visible in more than two images. By grouping nodes into connected components, an apple seen in, for example, five views is recovered as a single component of five nodes, even though no single matching step compared all five masks at once. The transitivity of the relationship between connected components transforms the pairwise matches into a consistent mapping across multiple views.



3.2.2 Union-Find

Computing the connected components of a graph via a disjoint-set data structure is a standard technique [5]. We use the *Union-Find* data structure, also known as the disjoint-set data structure, which manages a partition of the nodes into disjoint sets and efficiently supports two operations: `find`, which returns a representative element for the set containing a given node, and `union`, which merges the two sets containing two given nodes into one.

Each set is stored as a tree, in which every node points to a parent node, and the root of the tree serves as the representative of the set. The `find` operation follows the parent pointers from a node up to the root. Two nodes belong to the same set if and only if the `find` operation returns the same root for both. The `union` operation joins two sets by attaching the root of one tree as a child of the root of the other.

In our method, we initialize the Union-Find structure with a singleton set per node, i.e., one set for each instance mask in each view. For each edge that is established during the matching step, we call `union` for its two nodes. After all edges have been processed, the sets of the Union-Find structure are exactly the connected components of the graph, and each set represents a counted apple instance.

A naive implementation of `find` can degenerate when the trees become tall, since each call then traverses a long chain of parent pointers. To avoid this, we apply *path compression*: During a `find` operation, every node visited on the path to the root is immediately appended back to the root [5]. This makes repeated lookups on the same set progressively cheaper and keeps the extraction of connected components efficient even for the large number of instance nodes that arise across all views of a scene.

3.3 Hungarian Matching

The edges of the graph introduced in Section 3.2.1 are established by matching instance masks between pairs of views. We formulate this matching as a *assignment problem* and solve it using the Hungarian algorithm [19, 28]. Following the pointmap-based association of CCGS [16], the cost of a match is derived from the overlap of two masks in 3D; the details of this cost are given in Chapter 4, while this section introduces the assignment problem and the algorithm in general terms.

3.3.1 The Assignment Problem

The assignment problem is a classical combinatorial optimization problem. In its original formulation [19], a set of n workers is to be assigned n jobs, given a numerical rating for the performance of each worker on each job, such that the total rating of the chosen assignment is optimal and no worker is assigned more than one job. Our matching task has the same structure: Instead of worker and tasks, we assign the instance masks of one view to those of another, and instead of a performance score, each candidate pair carries a cost value that reflects how likely it is that the two masks represent the same apple.

Consider two views i and j , with instance masks indexed by $a \in \{1, \dots, n_i\}$ and $b \in \{1, \dots, n_j\}$ in views j . For every pair (a, b) , a cost $C_{ab} \geq 0$ expresses how unlikely it is that the two masks depict the same apple, where a lower cost indicates a stronger match. These costs form a cost matrix $C \in \mathbb{R}^{n_i \times n_j}$.

In its standard form, the linear sum assignment problem assumes an equal number of elements

on both sides and assigns each element exactly once. Describing an assignment by binary variables $x_{ab} \in \{0, 1\}$, where $x_{ab} = 1$ means that mask a is matched to mask b , it is written with equality constraints as $\sum_b x_{ab} = 1$ and $\sum_a x_{ab} = 1$, so that every element receives exactly one partner [2]. This assumption does not hold in our setting: The two views generally contain different numbers of masks, and a mask may have no counterpart at all when the apple it depicts is occluded in the other view. We therefore relax the equality constraints to inequalities, so that each mask is matched *at most* once and unmatched masks are permitted. The resulting problem is

$$\begin{aligned}
 \min_x \quad & \sum_a \sum_b C_{ab} x_{ab} \\
 \text{s.t.} \quad & \sum_b x_{ab} \leq 1 \quad \forall a, \\
 & \sum_a x_{ab} \leq 1 \quad \forall b, \\
 & x_{ab} \in \{0, 1\}.
 \end{aligned} \tag{3.1}$$

Because masks in view i are only ever paired with masks in view j , and never with masks in the same view, the underlying structure is a *bipartite* graph: its nodes split into the two disjoint sets of masks, and every candidate match connects one set to the other, with no edges inside either set. The cost matrix C is the weighted adjacency matrix of this bipartite graph, and the assignment problem is the search for a minimum-cost bipartite matching.

The fact that C is generally not square, and that some masks remain unmatched, is therefore not a defect but the intended behavior: an apple that is visible in one view but occluded in the other should not be assigned a partner. Each accepted match becomes an edge in the graph of Section 3.2.1, linking the two corresponding nodes.

3.3.2 The Hungarian Algorithm

The difficulty of the assignment problem is that the number of possible assignments grows factorially with the number of masks: for two views with n masks each, there are $n!$ distinct one-to-one assignments [28], so enumerating them and picking the cheapest is infeasible for all but the smallest cases. A tempting shortcut is a greedy strategy in which each mask is assigned to its individually best partner. Such a choice is locally optimal but can be globally suboptimal: two masks in view i might both prefer the same mask in view j , which violates the one-to-one requirement, and resolving the conflict greedily can force a worse overall assignment than necessary.

The Hungarian algorithm avoids both problems. Introduced by Kuhn [19], it is named for the underlying results of the Hungarian mathematicians D. König and E. Egerváry on which it is based, and it computes a globally optimal assignment without enumerating all possibilities. Munkres [28] later gave a variant of the method together with a proof that it terminates after a number of operations bounded by a polynomial in n , rather than the factorial cost of exhaustive search; because of this joint contribution the method is also known as the Kuhn-Munkres algorithm. It therefore solves the problem in Equation (3.1) exactly and efficiently. In our implementation, we obtain the optimal assignment using a standard solver for the linear assignment problem.

3.3.3 Partial Matching

The optimal assignment minimizes the total cost, but it does not by itself guarantee that the matched pairs are meaningful: the algorithm will continue to pair two masks whose overlap is negligible if doing so reduces the total cost. To prevent such spurious matches from becoming edges, we allow an assignment to be an edge only if its overlap exceeds a threshold and discard the remaining assignments. This corresponds to allowing *partial matches*, in which not every mask of the smaller view has to be matched, and follows the association strategy of CCGs[16]. The precise overlap measure and threshold are described in Chapter 4.

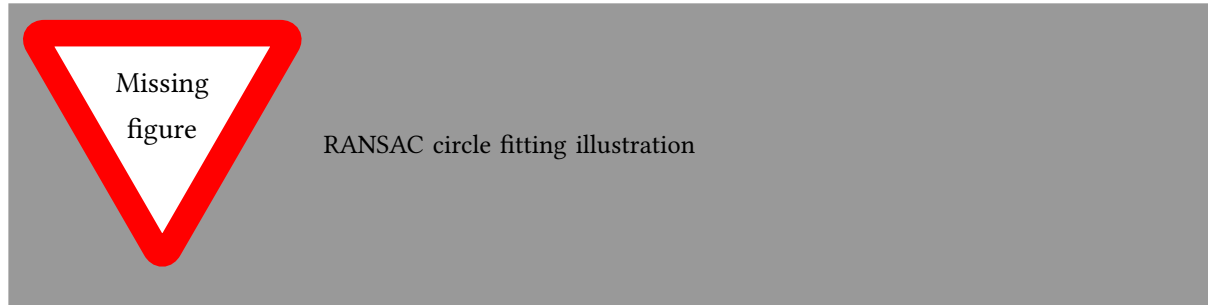
3.4 RANSAC

Fitting a geometric model to a set of measured points is a recurring subtask in our pipeline. Whenever such measurements originate from imperfect detectors or reconstructions, they contain not only small measurement noise but also *gross errors*: points that are simply wrong rather than slightly inaccurate. A conventional estimator such as least squares offers no protection against these. As an averaging technique, it relies on the assumption that any deviation from the model is a small measurement error and that such deviations balance each other out over a sufficiently large number of points [10]. Coarse errors clearly violate this assumption, and even a single severely misplaced point can distort the entire fit. To robustly estimate a model under such conditions, we use RANSAC (Random Sample Consensus).

RANSAC was introduced by Fischler and Bolles [10] and reverses the logic of conventional smoothing: Instead of fitting the model to all points and then attempting to remove those that do not match, it fits the model to a small sample and then measures how many of the remaining points match it. Each proceeds in three steps. First, it draws a *minimal sample*, the smallest number of points that suffices to instantiate the model, and computes the model from this sample alone. Second, it tests the instantiated model against all other points and collects every point that lies within a fixed error tolerance of it into the *consensus set* for this sample; points outside the tolerance are considered outliers. Third, the size of the consensus set is recorded as the value for this sample. These steps are repeated over a specified number of iterations, and the model with the largest consensus set is retained. Optionally, this final model is refitted to all points in its consensus set to refine the estimate [10].

The robustness of the method stems from this construction. Since the model is selected by consensus among many points rather than by minimizing a residual across all points, gross outliers that would derail an averaging estimator are excluded from the consensus set and therefore do not influence the retained model. Figure illustrates this on the simple case of fitting a circle to a set of 2D points: three points determine a candidate circle, and the points within the tolerance band form its consensus set, while the outliers that would bias a least-squares fit are left out.

figure
ransac



3.5 Clustering

The final step in our pipeline groups a set of points in 3D space so that points that belong together are combined into a single point and counted only once. Given a set of points, the task is to divide them into clusters so that points that are spatially close to one another belong to the same cluster, while points that are far apart belong to different clusters. The number of resulting clusters is the measure of interest. We consider two families of methods for this grouping.

3.5.1 Density-Based Clustering

An obvious first choice is DBSCAN, a density-based algorithm introduced by Ester et al. [9]. Its central idea is that clusters are regions of high point density separated from one another by regions of low density. Density is evaluated based on a neighborhood radius ϵ and a minimum number *MinPts*: A point is a *core point* if at least *MinPts* points lie within a distance of ϵ from it [9]. In practice, only ϵ needs to be chosen, since the authors report that *MinPts* can be set to a default value, so that the algorithm has only a single free parameter. Clusters are then formed from core points based on a concept of *density-reachability*: A point is reachable from a core point if it can be connected to it by a chain of points in which each successive point lies within ϵ of the previous one and is itself a core point [9]. A cluster is the maximum set of points that are densely connected to one another, and points that are not reachable from any core point are referred to as noise. This chaining allows DBSCAN to discover clusters of any shape, but it can be a disadvantage if clusters are expected to be spatially compact. Since membership propagates transitively along chains of ϵ -close points, two dense groups are kept separate from each other only if the distance between them is greater than ϵ [9]. If such groups are close to one another, the points of one may lie within ϵ of the other, and DBSCAN then connects them via the intervening points to a single cluster. Different groups are thereby linked into a single one. Reducing ϵ to break such chains is not a solution: it also separates loosely connected points within a single group, thereby splitting a cluster into several. In general, there is no single radius that separates neighboring groups while simultaneously uniting the points of each individual group.

3.5.2 Agglomerative Clustering with complete Linkage

We therefore use agglomerative hierarchical clustering, which controls the spatial extent of a cluster directly rather than via a density chain [14]

Chapter 4

Method

Chapter 5

Results

Describe your qualitative and quantitative evaluations and present your results. Discuss the limitations of your work.

Chapter 6

Conclusions

Give a brief summary of your work and your contributions. Afterwards, briefly outline potential opportunities for future work.

Todo list

citation for cost, other 3D capture methods	1
real-world data?	2
Figure: vgg pipeline figure	6
Figure: simply illustration	8
figure ransac	11
Figure: RANSAC circle fitting illustration	11

List of Figures

LIST OF FIGURES

List of Tables

LIST OF TABLES

List of Listings

LIST OF LISTINGS

Bibliography

- [1] Juan Ballena-Ruiz, Juan Arcila-Diaz, and Victor Tuesta-Monteza. “Automated Detection and Counting of Gossypium barbadense Fruits in Peruvian Crops Using Convolutional Neural Networks”. In: *AgriEngineering* 7.5 (2025), p. 152. ISSN: 2624-7402. DOI: 10.3390/agriengineering7050152.
- [2] Rainer Burkard, Mauro Dell’Amico, and Silvano Martello. *Assignment Problems*. Philadelphia, PA: Society for Industrial and Applied Mathematics (SIAM), 2009. ISBN: 978-0-898716-63-4. DOI: 10.1137/1.9780898717754.
- [3] Nicolas Carion et al. *SAM 3: Segment Anything with Concepts*. 2026. arXiv: 2511.16719 [cs.CV]. URL: <https://arxiv.org/abs/2511.16719>.
- [4] G. Ramesh Chandra, K. Sri Rama Murthy, S. Archana, B. Snehitha, G. Nishanth, M. Navya, and R. Sai Charan. “Fruit Detection and Counting Using Deep Learning”. In: *2025 Eighth International Conference on Image Information Processing (ICIIP)*. 2025, pp. 175–182. DOI: 10.1109/ICIIP68302.2025.11346118.
- [5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. 3rd ed. Cambridge, MA: MIT Press, 2009. ISBN: 978-0262033848.
- [6] Kai Deng, Zexin Ti, Jiawei Xu, Jian Yang, and Jin Xie. *VGGT-Long: Chunk it, Loop it, Align it – Pushing VGGT’s Limits on Kilometer-scale Long RGB Sequences*. 2025. arXiv: 2507.16443 [cs.CV]. URL: <https://arxiv.org/abs/2507.16443>.
- [7] Michiel van Dijk, Tom Morley, Marie Luise Rau, and Yashar Saghai. “A meta-analysis of projected global food demand and population at risk of hunger for the period 2010–2050”. In: *Nature Food* 2.7 (2021), pp. 494–501. DOI: 10.1038/s43016-021-00322-9.
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *International Conference on Learning Representations (ICLR)*. 2021.
- [9] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise”. In: *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*. Portland, OR: AAAI Press, 1996, pp. 226–231.
- [10] Martin A. Fischler and Robert C. Bolles. “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography”. In: *Communications of the ACM* 24.6 (1981), pp. 381–395. DOI: 10.1145/358669.358692.

BIBLIOGRAPHY

- [11] Jordi Gené-Mola, Ricardo Sanz-Cortiella, Joan R. Rosell-Polo, Josep-Ramon Morros, Javier Ruiz-Hidalgo, Verónica Vilaplana, and Eduard Gregorio. “Fruit detection and 3D location using instance segmentation neural networks and structure-from-motion photogrammetry”. In: *Computers and Electronics in Agriculture* 169 (2020), p. 105165. ISSN: 0168-1699. DOI: 10.1016/j.compag.2019.105165.
- [12] Nicolai Häni, Pravakar Roy, and Volkan Isler. “A comparative study of fruit detection and counting methods for yield mapping in apple orchards”. In: *Journal of Field Robotics* 37.2 (Aug. 2019), pp. 263–282. ISSN: 1556-4967. DOI: 10.1002/rob.21902. URL: <http://dx.doi.org/10.1002/rob.21902>.
- [13] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. 2nd ed. Cambridge, UK: Cambridge University Press, 2004. ISBN: 978-0521540513. DOI: 10.1017/CB09780511811685.
- [14] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York, NY: Springer, 2009. ISBN: 978-0-387-84857-0. DOI: 10.1007/978-0-387-84858-7.
- [15] Xiangyu Hong, Linlin Qin, Chun Shi, and Gang Wu. “FruitGaussian: 3D Gaussian Splatting for Automated Fruit Counting in Natural Orchard”. In: *Proceedings of the 44th Chinese Control Conference (CCC)*. 2025, pp. 8004–8009. DOI: 10.23919/CCC64809.2025.11179457.
- [16] Wenhao Hu, Wenhao Chai, Shengyu Hao, Xiaotong Cui, Xuexiang Wen, Jenq-Neng Hwang, and Gaoang Wang. “Pointmap Association and Piecewise-Plane Constraint for Consistent and Compact 3D Gaussian Segmentation Field”. In: *International Journal of Computer Vision* 134.6 (2026), p. 255. DOI: 10.1007/s11263-026-02821-3.
- [17] E. M. B. M. Karunathilake, Anh Tuan Le, Seong Heo, Yong Suk Chung, and Sheikh Mansoor. “The Path to Smart Farming: Innovations and Opportunities in Precision Agriculture”. In: *Agriculture* 13.8 (2023), p. 1593. DOI: 10.3390/agriculture13081593.
- [18] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. “3D Gaussian Splatting for Real-Time Radiance Field Rendering”. In: *ACM Transactions on Graphics* 42.4 (July 2023). URL: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>.
- [19] Harold W. Kuhn. “The Hungarian Method for the Assignment Problem”. In: *Naval Research Logistics Quarterly* 2.1–2 (Mar. 1955), pp. 83–97. DOI: 10.1002/nav.3800020109.
- [20] Vincent Leroy, Yohann Cabon, and Jerome Revaud. “Grounding Image Matching in 3D with MAST3R”. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2024.
- [21] Fengze Li, Yangle Liu, Jieming Ma, Hai-Ning Liang, Yaochun Shen, Huangxiang Li, and Zhijing Wu. *CountingFruit: Language-Guided 3D Fruit Counting with Semantic Gaussian Splatting*. 2025. arXiv: 2506.01109 [cs.CV].
- [22] Xu Liu, Steven W. Chen, Shreyas Aditya, Nivedha Sivakumar, Sandeep Dcunha, Chao Qu, Camillo J. Taylor, Jnaneshwar Das, and Vijay Kumar. *Robust Fruit Counting: Combining Deep Learning, Tracking, and Structure from Motion*. 2018. arXiv: 1804.00307 [cs.CV]. URL: <https://arxiv.org/abs/1804.00307>.

- [23] Xu Liu, Steven W. Chen, Chenhao Liu, Shreyas S. Shivakumar, Jnaneshwar Das, Camillo J. Taylor, James Underwood, and Vijay Kumar. “Monocular Camera Based Fruit Counting and Mapping With Semantic Data Association”. In: *IEEE Robotics and Automation Letters* 4.3 (2019), pp. 2296–2303. ISSN: 2377-3774. DOI: 10.1109/lra.2019.2901987. URL: <http://dx.doi.org/10.1109/LRA.2019.2901987>.
- [24] Dominic Maggio, Hyungtae Lim, and Luca Carlone. “VGGT-SLAM: Dense RGB SLAM Optimized on the SL (4) Manifold”. In: *Advances in Neural Information Processing Systems* 39 (2025).
- [25] Lukas Meyer, Andreas Gilson, Ute Schmid, and Marc Stamminger. “FruitNeRF: A Unified Neural Radiance Field based Fruit Counting Framework”. In: *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2024, pp. 1–8. DOI: 10.1109/IROS58592.2024.10802065.
- [26] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. “NeRF: representing scenes as neural radiance fields for view synthesis”. In: *Commun. ACM* 65.1 (Dec. 2021), pp. 99–106. ISSN: 0001-0782. DOI: 10.1145/3503250. URL: <https://doi.org/10.1145/3503250>.
- [27] Juan C. Miranda, Jordi Gené-Mola, Manuela Zude-Sasse, Nikos Tsoulas, Alexandre Escolà, Jaume Arnó, Joan R. Rosell-Polo, Ricardo Sanz-Cortiella, José A. Martínez-Casasnovas, and Eduard Gregorio. “Fruit sizing using AI: A review of methods and challenges”. In: *Postharvest Biology and Technology* 206 (2023), p. 112587. ISSN: 0925-5214. DOI: 10.1016/j.postharvbio.2023.112587.
- [28] James Munkres. “Algorithms for the Assignment and Transportation Problems”. In: *Journal of the Society for Industrial and Applied Mathematics* 5.1 (1957), pp. 32–38. DOI: 10.1137/0105003. eprint: <https://doi.org/10.1137/0105003>. URL: <https://doi.org/10.1137/0105003>.
- [29] Maxime Oquab et al. “DINOv2: Learning Robust Visual Features without Supervision”. In: *Transactions on Machine Learning Research* (2024). ISSN: 2835-8856.
- [30] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. “Vision Transformers for Dense Prediction”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021.
- [31] Johannes L. Schönberger and Jan-Michael Frahm. “Structure-from-Motion Revisited”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 4104–4113. DOI: 10.1109/CVPR.2016.445.
- [32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017.
- [33] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. “VGGT: Visual Geometry Grounded Transformer”. In: *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025, pp. 5294–5306. DOI: 10.1109/CVPR52734.2025.00499.
- [34] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. “DUST3R: Geometric 3D Vision Made Easy”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2024, pp. 20697–20709.

BIBLIOGRAPHY

- [35] Yifan Wang, Jianjun Zhou, Haoyi Zhu, Wenzheng Chang, Yang Zhou, Zizun Li, Junyi Chen, Jiangmiao Pang, Chunhua Shen, and Tong He. “ π^3 : Permutation-Equivariant Visual Geometry Learning”. In: *arXiv preprint arXiv:2507.13347* (2025).
- [36] Mi Yan, Jiazhao Zhang, Yan Zhu, and He Wang. “MaskClustering: View Consensus Based Mask Graph Clustering for Open-Vocabulary 3D Instance Segmentation”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024, pp. 28274–28284.
- [37] Jianing Yang, Alexander Sax, Kevin J. Liang, Mikael Henaff, Hao Tang, Ang Cao, Joyce Chai, Franziska Meier, and Matt Feiszli. “Fast3R: Towards 3D Reconstruction of 1000+ Images in One Forward Pass”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2025.

Declaration

I certify that I have written this thesis without outside help and without using sources other than those indicated, and that the thesis has not been submitted in the same or similar form to any other examination authority and has not been accepted by them as part of an examination. All statements that have been taken over verbatim or in spirit are marked as such.

During the preparation of this work, AI technologies were used to assist in the writing process. Specifically, <AI Tool 1 (Company, City, State, Country)> was used in order to check for grammar and style consistency, and <AI Tool 2 (Company, City, State, Country)> was used in order to assist with rephrasing and improving readability. After using these tools, the manuscript was carefully reviewed and the content was edited as needed. No tools or services were used for content generation. The content of this thesis was entirely created by myself, and I ensured that all material presented reflects my original work and research. I take full responsibility for the content of the thesis.

I agree that the thesis may be published and that it may be referred to in scientific publications.

The Friedrich-Alexander-Universität Erlangen-Nürnberg, represented by the Professorship for Cognitive Computer Vision, is granted a (non-exclusive) right to use this work and the programs created in connection with it.

Erlangen, 01.10.2026

(Alexander Link)